A1.1 — Dataset và Protocol thí nghiệm

1. Tổng quan

Nhóm so sánh CNN và Transformer trên Food-101 (101 lớp món ăn), mỗi mô hình huấn luyện theo 4 chế độ: from-scratch, freeze toàn bộ backbone, freeze một phần và full finetune.

Cặp chính, cặp mở rộng và số run. Một run là một lần huấn luyện một mô hình theo một chế độ. Mỗi mô hình chạy 4 chế độ, nên một cặp gồm 1 CNN và 1 Transformer cho 2 × 4 = 8 run.

- **Cặp chính (bắt buộc):** ResNet-50 và DeiT-S, 8 run đánh số R1–R8. Đây là phần đáp ứng yêu cầu của đề.
- **Cặp mở rộng (tùy chọn):** ConvNeXt-T và Swin-T, 8 run đánh số X1–X8. Làm thêm nếu còn thời gian, để kiểm tra kết luận của cặp chính trên hai kiến trúc hiện đại hơn.

Mục	Lựa chọn
Môn học	CO5085 — Học sâu và ứng dụng trong thị giác máy tính, HK261
Giảng viên	Lê Thành Sách, Nguyễn Quốc Minh
Tập dữ liệu	Food-101
Cặp chính (bắt buộc)	ResNet-50 (CNN) vs DeiT-Small (ViT thuần)
Cặp mở rộng (tùy chọn)	ConvNeXt-Tiny (CNN) vs Swin-Tiny (Transformer phân cấp)
Số run	8 run bắt buộc (cặp chính, R1–R8) + 8 run tùy chọn (cặp mở rộng, X1–X8)

Câu hỏi cần trả lời:

1. Pretrain giúp CNN và ViT đến mức nào trên một tập khoảng 75k ảnh train?
2. Mức freeze nào cân bằng tốt nhất giữa accuracy và chi phí (thời gian, tham số trainable, bộ nhớ)?
3. (Mở rộng) Transformer phân cấp như Swin có bớt phụ thuộc vào pretrain hơn ViT thuần không?

2. Tập dữ liệu: Food-101

Food-101 gồm 101.000 ảnh thuộc 101 lớp món ăn, mỗi lớp 1.000 ảnh, do ETH Zurich công bố tại ECCV 2014 (trang chính thức, Hugging Face).

Thuộc tính	Giá trị
Số lớp	101, cân bằng (1.000 ảnh/lớp)
Train chính thức	75.750 ảnh (750/lớp)
Test chính thức	25.250 ảnh (250/lớp)
Độ phân giải	Ảnh màu, cạnh dài nhất tối đa 512 px
Dung lượng	Khoảng 5 GB (file nén gốc)
Nhãn	Test được kiểm tra thủ công; train cố ý để nhiễu (màu gắt, đôi khi sai nhãn)
Nguồn tải	`torchvision.datasets.Food101` hoặc `ethz/food101` trên Hugging Face

License. Ảnh lấy từ Foodspotting, không thuộc sở hữu của ETH Zurich. Việc sử dụng trong phạm vi nghiên cứu, học tập (scientific fair use) được phép; mọi mục đích khác phải thỏa thuận với chủ ảnh. Bài tập này chỉ dùng cho học tập, không phân phối lại ảnh.

Lý do chọn.

- Đủ lớn và đủ khó: 101 lớp, 75k ảnh train, đúng phạm vi "vài nghìn đến chục nghìn ảnh" của đề.
- Phân loại fine-grained: nhiều món giống nhau về màu và kết cấu, nên khác biệt giữa các mô hình và chế độ huấn luyện sẽ thể hiện rõ.
- Ảnh thật, độ phân giải cao: chạy được ở 224×224 đúng chuẩn của các mô hình pretrained, không phải phóng to ảnh nhỏ như CIFAR.
- Cân bằng lớp và có split test chuẩn, nên dễ so sánh với kết quả đã công bố.
- Train nhiễu nhãn là một điểm thảo luận thêm khi phân tích lỗi.

3. Split và tiền xử lý

Food-101 đã có sẵn tập test chính thức, nên nhóm giữ nguyên tập test này và chỉ tách 10% ảnh train làm validation, chia đều theo từng lớp (stratified), seed cố định 42. Mọi run dùng chung một split.

Split	Nguồn	Ảnh/lớp	Tổng	Dùng để
Train	90% train chính thức	675	68.175	Huấn luyện
Validation	10% train chính thức	75	7.575	Chọn checkpoint, chỉnh siêu tham số
Test	Test chính thức	250	25.250	Báo cáo kết quả cuối, chỉ đánh giá một lần

- Danh sách chỉ số ảnh của train/validation lưu vào `A1/splits/` trong repo để tái lập.
- Validation lấy từ train nên cũng có nhiễu nhãn; đánh giá chính xác nhất vẫn là test đã được kiểm tra thủ công.

- Không dùng subset: Colab A100 đủ để chạy toàn bộ 75.750 ảnh cho tất cả các run. Nếu phải cắt giảm, sẽ lấy subset theo tỷ lệ cố định mỗi lớp và ghi rõ trong báo cáo.

Tiền xử lý chung. Ảnh đưa về 224×224, chuẩn hóa bằng mean/std của ImageNet. Áp dụng cho cả run from-scratch để mọi run nhận đầu vào giống hệt nhau.

Phân tích khám phá dữ liệu (Exploratory Data Analysis) sẽ làm ở A1.2. Đây là bước khảo sát dữ liệu trước khi huấn luyện để hiểu đặc điểm của tập: phân bố kích thước ảnh, lưới ảnh mẫu theo lớp, các cặp lớp dễ nhầm, và ví dụ ảnh train sai nhãn.

4. Augmentation

Mức augmentation tăng theo số tham số được train; trong cùng một chế độ, CNN và Transformer dùng cùng một pipeline để so sánh công bằng.

Augmentation là biến đổi ngẫu nhiên ảnh train (cắt, lật, đổi màu...) để mô hình thấy nhiều biến thể hơn và bớt overfit. Biến đổi diễn ra trực tiếp lúc train: mỗi epoch, mỗi ảnh bị biến đổi khác nhau, không tạo thêm file ảnh trên ổ đĩa. Đề không quy định kỹ thuật cụ thể; nhóm chọn các kỹ thuật phổ biến cho ResNet và ViT.

Ý nghĩa từng kỹ thuật:

- **RandomResizedCrop 224:** cắt một vùng ngẫu nhiên (kích thước và tỷ lệ khung khác nhau) rồi đưa về 224×224, giúp mô hình nhận ra món ăn ở nhiều vị trí và mức phóng to.
- **HorizontalFlip:** lật ngang ảnh với xác suất 50%; món ăn lật ngang vẫn là món đó.
- **RandAugment (n=2, m=9):** mỗi ảnh áp ngẫu nhiên 2 phép biến đổi (xoay, đổi độ sáng, độ tương phản, độ nét...) với cường độ 9 trên thang 30.
- **RandomErasing (p=0,25):** với xác suất 25%, xóa một ô chữ nhật ngẫu nhiên trong ảnh, buộc mô hình không phụ thuộc vào một vùng duy nhất.
- **Mixup (α=0,8):** trộn pixel của hai ảnh theo một tỷ lệ ngẫu nhiên, nhãn cũng trộn theo đúng tỷ lệ đó.
- **CutMix (α=1,0):** dán một mảng của ảnh này vào ảnh kia, nhãn trộn theo diện tích mảng được dán. Mỗi batch chọn ngẫu nhiên Mixup hoặc CutMix.
- **Label smoothing (0,1):** thay nhãn one-hot bằng nhãn "mềm" (lớp đúng 0,9, phần còn lại chia đều cho các lớp khác), giảm việc mô hình quá tự tin. Đây là kỹ thuật trên nhãn chứ không trên ảnh, nhưng dùng cùng mục đích chống overfit.

Chế độ	Augmentation khi train
Freeze toàn bộ backbone	RandomResizedCrop 224, HorizontalFlip
Freeze một phần, full finetune	Như trên + RandAugment (n=2, m=9), RandomErasing (p=0,25)
From-scratch	Như trên + Mixup (α=0,8) và CutMix (α=1,0), label smoothing 0,1

Khi đánh giá (validation, test): Resize cạnh ngắn về 256, CenterCrop 224, chuẩn hóa. Không dùng test-time augmentation.

Lý do augmentation mạnh cho from-scratch: ViT thuần không có inductive bias không gian nên dễ overfit trên 68k ảnh; công thức này theo hướng của DeiT. Áp dụng cùng công thức cho ResNet-50 để khác biệt chỉ đến từ kiến trúc.

5. Mô hình

Cặp chính là ResNet-50 vs DeiT-Small; cặp mở rộng ConvNeXt-Tiny vs Swin-Tiny làm nếu còn thời gian.

Thư viện timm. Mọi mô hình lấy từ timm (PyTorch Image Models), thư viện mã nguồn mở hiện do Hugging Face duy trì. timm cung cấp hàng trăm kiến trúc phân loại ảnh kèm trọng số pretrained; chỉ một dòng `timm.create_model("<tên mô hình>", pretrained=True, num_classes=101)` là có mô hình đã thay head 101 lớp. Nhóm dùng bản pretrained ImageNet-1k (không dùng bản ImageNet-22k lớn hơn) để mọi mô hình có cùng dữ liệu pretrain.

Các cột trong bảng. Tham số là số trọng số của mô hình. GFLOPs là số phép tính (tỷ phép) để xử lý một ảnh 224×224, đại diện cho chi phí tính toán. Top-1 ImageNet-1k là tỷ lệ ảnh mà dự đoán đầu tiên của mô hình đúng trên ImageNet, cho biết chất lượng của trọng số pretrained.

Vì sao cặp chính là ResNet-50 vs DeiT-S. Hai mô hình cùng cỡ (22–26M tham số, ~4 GFLOPs). DeiT-S là ViT thuần: attention toàn cục, không có inductive bias không gian như convolution. Vì vậy cặp này cho thấy rõ nhất câu hỏi "khi nào ViT cần pretrain".

Vì sao cặp mở rộng là ConvNeXt-T vs Swin-T. ConvNeXt được thiết kế bằng cách đưa ResNet theo hướng của Swin, nên hai mô hình gần như bằng nhau về cỡ và cách huấn luyện. Swin dùng attention cục bộ theo cửa sổ và cấu trúc phân cấp, nên có thể bớt phụ thuộc pretrain hơn DeiT. So sánh hai cặp cho biết khác biệt đến từ "convolution vs attention" hay từ "có hay không có inductive bias cục bộ".

Head phân loại. Thay head ImageNet bằng một lớp Linear 101 lớp (khởi tạo mới), giống nhau cho mọi mô hình.

Mô hình	Cặp	Loại	Tham số	GFLOPs (224)	Top-1 ImageNet-1k	Tên trong timm
ResNet-50	Chính	CNN	25,6M	4,1	~80% (recipe A1 của timm)	`resnet50.a1_in1k`
DeiT-Small	Chính	ViT thuần	22,1M	4,6	79,8%	`deit_small_patch16_224.fb_in1k`
ConvNeXt-Tiny	Mở rộng	CNN hiện đại	28,6M	4,5	82,1%	`convnext_tiny.fb_in1k`

Mô hình	Cặp	Loại	Tham số	GFLOPs (224)	Top-1 ImageNet-1k	Tên trong timm
Swin-Tiny	Mở rộng	Transformer phân cấp	28,3M	4,5	81,3%	swin_tiny_patch4_window7_224.ms_in1k

Số liệu tham số, FLOPs và top-1 lấy gần đúng theo các paper gốc; sẽ kiểm tra lại bằng timm khi chạy. Tham số trên chưa tính head mới 101 lớp.

6. Ma trận thí nghiệm

Cặp chính gồm 8 run: 2 mô hình × 4 chế độ, chạy from-scratch cho cả hai họ theo khuyến nghị của đề.

Định nghĩa "freeze một phần". Đóng băng các tầng đầu (đặc trưng cấp thấp như cạnh, kết cấu) và train các tầng sau, sao cho khoảng 50–60% tham số backbone được train:

- ResNet-50: freeze stem + layer1–layer3, train layer4 (~60% tham số).
- DeiT-S: freeze patch embedding, position embedding, CLS token và block 0–5, train block 6–11 (~50% tham số).
- ConvNeXt-T, Swin-T (mở rộng): freeze stem + stage 0–2, train stage cuối (~50% tham số).

Chia theo tỷ lệ tham số thay vì theo số stage, vì với ResNet-50, freeze 2 stage đầu chỉ khóa chưa đến 10% tham số và gần như không khác full finetune.

BatchNorm khi freeze. Với ResNet-50 ở R2 và R3, các lớp BatchNorm bị đóng băng giữ ở chế độ eval() (không cập nhật running mean/var), để "freeze" đúng nghĩa.

ID	Mô hình	Khởi tạo	Phần được train	Tham số trainable (ước tính)
R1	ResNet-50	Ngẫu nhiên	Toàn bộ	~23,7M
R2	ResNet-50	Pretrained	Chỉ head	~0,2M
R3	ResNet-50	Pretrained	layer4 + head	~15,2M
R4	ResNet-50	Pretrained	Toàn bộ	~23,7M
R5	DeiT-S	Ngẫu nhiên	Toàn bộ	~21,7M
R6	DeiT-S	Pretrained	Chỉ head	~0,04M
R7	DeiT-S	Pretrained	Block 6–11 + norm cuối + head	~10,7M
R8	DeiT-S	Pretrained	Toàn bộ	~21,7M

Số tham số trainable là ước tính; sẽ đếm chính xác bằng code (`p.requires_grad`) và ghi vào bảng kết quả.

Cặp mở rộng (X1–X8). Lặp lại đúng 4 chế độ trên cho ConvNeXt-T (X1–X4) và Swin-T (X5–X8), cùng split, augmentation và siêu tham số.

7. Siêu tham số và cấu hình huấn luyện

Mọi run dùng AdamW, lịch learning rate cosine có warmup, batch 256, mixed precision bf16; chỉ số epoch và learning rate thay đổi theo chế độ.

Các thuật ngữ trong bảng:

- **Epoch:** một lượt đi qua toàn bộ tập train. **Batch:** số ảnh xử lý cùng lúc trong một bước cập nhật.
- **AdamW:** thuật toán tối ưu phổ biến cho cả CNN và Transformer, tách riêng weight decay khỏi bước cập nhật gradient.
- **LR (learning rate):** độ lớn mỗi bước cập nhật trọng số. Head mới khởi tạo nên dùng LR lớn hơn backbone đã pretrained.
- **Cosine và warmup:** LR tăng dần từ gần 0 trong vài epoch đầu (warmup) để tránh làm hỏng trọng số pretrained, sau đó giảm dần theo đường cosine về gần 0.
- **Weight decay:** phạt trọng số lớn để giảm overfit.
- **Drop path:** ngẫu nhiên bỏ qua cả một block trong lúc train, một kỹ thuật chống overfit dùng cho mạng sâu.
- **bf16 (mixed precision):** tính toán bằng số thực 16 bit để train nhanh hơn và tốn ít bộ nhớ hơn, gần như không ảnh hưởng độ chính xác.

Chế độ	Epoch	LR backbone	LR head	Warmup	Weight decay	Drop path
From-scratch	100	5e-4	5e-4	5 epoch	0,05	0,1
Freeze toàn bộ backbone	30	—	1e-3	0	0	—
Freeze một phần	15	1e-4	1e-3	2 epoch	0,05	0,1
Full finetune	15	1e-4	1e-3	2 epoch	0,05	0,1

- **Công bằng khi chỉnh tham số:** với full finetune, thử LR backbone trong {5e-5, 1e-4, 3e-4} trên validation, cùng ngân sách cho cả hai kiến trúc, rồi dùng giá trị tốt nhất cho chế độ freeze một phần.
- **Chọn checkpoint:** lấy epoch có validation accuracy cao nhất; test chỉ đánh giá trên checkpoint đó.
- **Số seed:** các run pretrained của cặp chính (R2–R4, R6–R8) chạy 3 seed (42, 43, 44), báo cáo mean ± std. Run from-scratch (R1, R5) và cặp mở rộng chạy 1 seed; thêm seed nếu còn compute unit.
- **Vòng lặp huấn luyện:** tự viết bằng PyTorch (forward, loss, backward, optimizer step), không dùng `Trainer.fit()`.

8. Metric và kế hoạch phân tích

Metric chính là top-1 accuracy trên test; mỗi run còn ghi đủ các chỉ số về độ chính xác và chi phí dưới đây.

Các metric trong bảng:

- **Top-1 accuracy:** tỷ lệ ảnh mà lớp được dự đoán cao nhất là lớp đúng. **Top-5 accuracy:** tỷ lệ ảnh mà lớp đúng nằm trong 5 dự đoán cao nhất.
- **Macro-F1:** tính F1 (trung bình điều hòa của precision và recall) cho từng lớp rồi lấy trung bình, để mọi lớp có trọng số như nhau.
- **Confusion matrix:** bảng đếm số ảnh của lớp thật i bị dự đoán thành lớp j, cho thấy các cặp lớp hay bị nhầm.

Nhóm	Metric	Cách đo
Độ chính xác	Top-1, top-5 accuracy	Trên validation mỗi epoch; trên test một lần với checkpoint tốt nhất
Độ chính xác	Macro-F1	Trên test, bằng sklearn
Chi phí	Tham số trainable / tổng	Đếm p.numel() theo requires_grad
Chi phí	Thời gian mỗi epoch và tổng thời gian train	Đo bằng wall-clock, ghi kèm loại GPU
Chi phí	Bộ nhớ GPU đỉnh	torch.cuda.max_memory_allocated()
Chi phí	Tốc độ suy luận	Ảnh/giây trên test, batch 256

Biểu đồ và phân tích sẽ có trong A1.2:

1. Bảng tổng hợp 8 run (mean ± std với các run 3 seed).
2. Đường cong loss và accuracy theo epoch của từng run.
3. Biểu đồ accuracy theo số tham số trainable và theo thời gian train, để thấy đánh đổi giữa các mức freeze.
4. Khoảng cách pretrained − from-scratch của CNN so với của Transformer, để trả lời "ViT có cần pretrain hơn CNN không".
5. Confusion matrix của run tốt nhất mỗi mô hình, kèm 10 cặp lớp hay bị nhầm nhất.
6. Lưới ảnh dự đoán sai, phân loại nguyên nhân (món giống nhau, ảnh nhiều món, nhãn sai).
7. (Tùy chọn) Grad-CAM hoặc attention map để so vùng ảnh mà CNN và ViT dựa vào.

9. Phần cứng, tái lập và cấu trúc repo

Mọi run chính thức chạy trên Google Colab (gói trả phí), GPU A100, để thời gian huấn luyện giữa các run so sánh được với nhau.

- **GPU:** A100 cho mọi run đưa vào bảng kết quả. Khi Colab không cấp A100, chỉ dùng L4/T4 để debug; nếu buộc phải chạy run chính trên GPU khác, ghi rõ loại GPU và không so thời gian của run đó với các run A100.
- **Dữ liệu:** copy Food-101 vào ổ cục bộ `/content` ở đầu mỗi phiên, không đọc trực tiếp từ Google Drive.
- **Chống mất tiến độ:** lưu checkpoint lên Drive sau mỗi epoch và hỗ trợ chạy tiếp (resume) khi phiên bị ngắt.
- **Freeze toàn bộ backbone:** trích feature một lần rồi train head trên feature đã lưu, để tiết kiệm compute unit.
- **Tái lập:** cố định seed, lưu config YAML và log của từng run, ghim phiên bản PyTorch và `timm` trong `requirements.txt`.
- **Ngân sách ước tính:** cặp chính khoảng 20–25 giờ A100 (gồm 3 seed cho run pretrained và dự phòng), cặp mở rộng thêm khoảng 10 giờ. Sẽ đo tốc độ tiêu thụ compute unit thực tế ở epoch đầu rồi điều chỉnh số seed nếu cần.

Cấu trúc thư mục dự kiến:

```
A1/
├── README.md          # môi trường, lệnh chạy từng run
├── requirements.txt
├── configs/           # 1 file YAML cho mỗi run (R1-R8, X1-X8)
├── splits/            # chỉ số ảnh train/validation
├── src/
│   ├── data.py        # Dataset, DataLoader, augmentation
│   ├── models.py      # tạo mô hình timm, hàm freeze
│   ├── train.py       # vòng lặp huấn luyện tự viết
│   └── evaluate.py    # metric, confusion matrix, ảnh lỗi
├── notebooks/         # notebook Colab để chạy và EDA
└── results/           # log, bảng số liệu, biểu đồ
```

Nguồn tham khảo

- Bossard, Guillaumin, Van Gool. *Food-101 – Mining Discriminative Components with Random Forests*. ECCV 2014. Trang dataset
- Food-101 trên Hugging Face
- He và cộng sự. *Deep Residual Learning for Image Recognition*. CVPR 2016.
- Touvron và cộng sự. *Training data-efficient image transformers & distillation through attention* (DeiT). ICML 2021.
- Liu và cộng sự. *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. ICCV 2021.
- Liu và cộng sự. *A ConvNet for the 2020s* (ConvNeXt). CVPR 2022.